upGrad

Assignment Workflow
Hybrid RAG & Fine-Tuning for Customer Support

Introduction

This assignment focuses on developing a controlled customer support generation system using open-source Large Language Models (LLMs), retrieval-assisted generation (RAG), and parameter-efficient fine-tuning techniques.

You will design and implement a complete end-to-end machine learning workflow that begins with understanding business objectives, analysing and preparing instruction-tuning datasets (customer interactions) and knowledge bases (corporate SOPs), and establishing a baseline performance benchmark using a pre-trained language model. The project then progresses through the design and implementation of a retrieval-assisted generation system that retrieves semantically similar corporate policies to enhance model outputs.

Following the evaluation of the naive retrieval-based system, you will implement parameter-efficient fine-tuning using LoRA or QLoRA to train the model to extract structured JSON intents from noisy queries. This fine-tuned intent router is integrated with the existing retrieval pipeline. The final solution is evaluated against both the baseline model and the naive retrieval system to measure the impact of fine-tuning and retrieval independently.

The three progressively enhanced systems are:

1. Baseline pre-trained model
2. Retrieval-assisted generation system (Naive RAG)
3. Fine-tuned retrieval-assisted generation system (Hybrid RAG)

Business Problem

Modern organisations increasingly rely on automated customer support systems to handle repetitive inquiries, accelerate resolution times, improve productivity, and support technical decision-making. While general-purpose language models can hold conversations, they frequently produce inconsistent outputs, hallucinate refund windows, invent non-existent company policies, or fail to accurately classify the user's core intent.

Retrieval-assisted generation improves output quality by providing relevant contextual examples (Standard Operating Procedures) during inference. However, retrieval alone may not fully address domain-specific requirements: naive semantic search often fails when customer queries are messy, ambiguous, or sarcastic. Organisations therefore combine retrieval with parameter-efficient fine-tuning to accurately extract the user's intent first, using that structured data to drive the retrieval pipeline.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Data Description

The project uses a hybrid dataset approach designed to support intent extraction and factual generation tasks.

1. The Instruction-Tuning Dataset (Customer Interactions)

Sourced from the bitext/Bitext-customer-support-llm-chatbot-training-dataset. Each record must contain, or be transformed into: Prompt (Raw Customer Query) and Code Solution (Target JSON containing Intent and Category).

2. The Retrieval Knowledge Base (Corporate Policies)

A directory of Markdown (.md) files containing operational rules, including at least: refund_policy.md, shipping_delays.md, password_reset.md, technical_troubleshooting.md, escalation_matrix.md.

3. The Evaluation Datasets (Held-Out Test Split)

The held-out test partition (created via stratified train_test_split in Stage 2) serves as the primary evaluation set, guaranteeing zero data leakage. An adversarial subset is derived from the test split via regex filtering for sentiment/hedging language (e.g., 'still', 'never', 'terrible'). External CSV files may optionally supplement evaluation but require a programmatic leakage audit before scoring.

Prior to model development, you must assess dataset completeness; formatting consistency; duplicate records; near-duplicate records (detected via normalised lowercase + strip deduplication); prompt-solution alignment (intent distribution and class imbalance); and policy-document cosine similarity (TF-IDF overlap).

The instruction dataset must contain between 1,000 and 5,000 valid records after cleaning. The dataset must be partitioned into Training, Validation, and Testing sets (recommended 80/10/10 or 70/15/15). Data-leakage prevention must be demonstrated by ensuring no duplicate prompts appear across partitions (verified programmatically via set-intersection checks), near-duplicates are addressed, evaluation records are never used in training, and all three partitions remain independent.

Overall Methodology and Learning Outcomes

The project follows a structured ML/LLM development lifecycle emphasising reproducibility, systematic experimentation, objective evaluation, and evidence-based decision-making, organised into four stages.

Stage 1 — Business Objective, Data Understanding, and Proposed Methodology

Define project scope, inspect the dataset, perform EDA (intent balance, token distributions, TF-IDF cosine similarity), and design a complete modelling methodology with quantitative success criteria defined before development begins.

Stage 2 — Data Preparation

The dataset is cleaned via normalised deduplication (lowercase + strip before drop_duplicates), standardised, transformed into ChatML with strict JSON targets, tokenised with label masking, analysed for sequence length, and partitioned via stratified sampling (stratify=intent). The Markdown retrieval corpus is prepared and chunked.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Stage 3 — Solution V1 (Retrieval-Assisted Generation)

A baseline evaluation establishes a reference point. Then implement a naive RAG pipeline (embedding generation, ChromaDB index, similarity search, prompt augmentation) and evaluate it using the same framework to determine retrieval's independent impact.

Stage 4 — Solution V2 (Fine-Tuned Retrieval-Assisted Generation)

Configure a PEFT pipeline (LoRA/QLoRA), train with a custom AdamW loop, monitor validation loss with early stopping (saving the best checkpoint), plot loss curves, and evaluate (Format Adherence, Exact Match, Fuzzy Match on the held-out test split and adversarial subset). The intent-routing model is integrated into the retrieval pipeline and the full Hybrid RAG system is compared against Baseline and V1.

Evaluation Metrics

- Format Adherence Rate (%): whether generated output is syntactically valid JSON (Solution V2 router).
- Intent Accuracy Rate (%): exact-match accuracy of extracted intents.
- ROUGE-1, ROUGE-L, and BLEU: semantic similarity of the final response against SOP-grounded references.
- Consistency Rate (%): stability across repeated generations using deterministic inference (do_sample=False with temperature=None).
- Hallucination Frequency (%): percentage of outputs containing policy violations or fabricated facts (Baseline, V1, V2).

Project Stages

Stage 1 — Business Objective, Data Understanding, and Proposed Methodology [25 Marks]

Task 1.1 — Define Project Scope [Deliverable: Project Proposal PDF] [5 Marks]

- Subtask 1.1.1 — Identify the Target Use Case: Select a single support category; define expected JSON inputs/outputs and project boundaries. [3 Marks]
- Subtask 1.1.2 — Define Success Criteria: Establish measurable objectives and evaluation targets (Format Adherence, ROUGE/BLEU, Hallucination Reduction). [2 Marks]

Task 1.2 — Understand and Assess Data [Deliverable: EDA Notebook] [7 Marks]

- Subtask 1.2.1 — Inspect Dataset Structure: Load datasets; review Bitext schema and SOP structure; assess completeness. [2 Marks]
- Subtask 1.2.2 — Quantify Dataset Characteristics: Analyse dataset size, prompt lengths, and token distributions; downsample to the 1,000-5,000 range. [3 Marks]
- Subtask 1.2.3 — Validate Sample Quality: Detect incomplete records, formatting inconsistencies, duplicates, and near-duplicates (normalised deduplication); verify prompt-solution alignment. [2 Marks]

Task 1.3 — Perform Exploratory Data Analysis [Deliverable: EDA Notebook] [6 Marks]

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

- Subtask 1.3.1 — Perform Univariate Analysis: Analyse intent class balance and length characteristics. [2 Marks]
- Subtask 1.3.2 — Perform Bivariate Analysis: TF-IDF cosine-similarity checks on SOP documents to detect conflicting rules. [2 Marks]
- Subtask 1.3.3 — Generate Visualisations: Create distribution plots and similarity heatmaps and interpret findings. [2 Marks]

Task 1.4 — Design Methodology [Deliverable: Project Proposal PDF] [7 Marks]

- Subtask 1.4.1 — Select Baseline Model: Justify using hardware constraints, dataset characteristics, and fine-tuning feasibility. [2 Marks]
- Subtask 1.4.2 — Design Retrieval Strategy: Define embedding, retrieval workflow, similarity search, vector storage, and retrieval evaluation. [2 Marks]
- Subtask 1.4.3 — Design Fine-Tuning Strategy: Define LoRA/QLoRA strategy, target modules, ChatML structure, checkpointing, and logging. [2 Marks]
- Subtask 1.4.4 — Define Evaluation Framework: Define procedures for Format Adherence, Intent Accuracy, Consistency, and Hallucination Frequency. [1 Mark]

Stage 2 — Data Preparation [Deliverable: Data Preparation Notebook] [20 Marks]

Task 2.1 — Clean and Validate Dataset [4 Marks]

- Subtask 2.1.1 — Remove Duplicate and Invalid Records: Identify and remove duplicate records using normalised deduplication (lowercase + strip before comparison), invalid records, incomplete records, and corrupted records. [2 Marks]
- Subtask 2.1.2 — Standardise Dataset Formatting: Standardise prompt structure, casing (intent lowercase, category uppercase), and dataset schema. [2 Marks]

Task 2.2 — Create Instruction-Tuning Dataset [4 Marks]

- Subtask 2.2.1 — Transform Records into Instruction Format: Convert records into ChatML with strict JSON targets. [2 Marks]
- Subtask 2.2.2 — Validate Instruction Templates: Verify formatting consistency, validate template correctness, and verify schema compliance. [2 Marks]

Task 2.3 — Engineer Training and Retrieval Data [7 Marks]

- Subtask 2.3.1 — Tokenise Dataset: Configure tokenizer, generate model-ready sequences, apply label masking (-100). [2 Marks]
- Subtask 2.3.2 — Analyse Token Distributions: Analyse token counts, sequence lengths, and truncation risks. [2 Marks]
- Subtask 2.3.3 — Prepare Retrieval Corpus: Create retrieval-ready records (chunk Markdown files) and validate corpus quality. [3 Marks]

Task 2.4 — Create Dataset Partitions [5 Marks]

- Subtask 2.4.1 — Create Train, Validation, and Test Splits: Generate representative training, validation, and testing partitions using stratified sampling (stratify=intent) so all intents appear in every split. The validation set must be actively used during fine-tuning (Stage 4) for loss monitoring and early stopping. Demonstrate no duplicate prompts across partitions, no evaluation leakage, and three independent datasets via programmatic set-intersection checks. [5 Marks]

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Stage 3 — Solution V1 (Retrieval-Assisted Generation) [30 Marks]

Task 3.1 — Establish Baseline Performance [Deliverable: Baseline Model Evaluation Notebook] [4 Marks]

- Subtask 3.1.1 — Execute Baseline Inference: Load baseline model, execute deterministic inference, generate baseline outputs. [2 Marks]
- Subtask 3.1.2 — Evaluate Baseline Quality: Evaluate functional correctness, output consistency, and hallucination frequency. [2 Marks]

Task 3.2 — Implement Retrieval-Assisted Generation [Deliverable: RAG Implementation Notebook] [12 Marks]

- Subtask 3.2.1 — Generate Embeddings: Implement embedding generation using all-MiniLM-L6-v2, embedding storage, and embedding validation. [4 Marks]
- Subtask 3.2.2 — Build Vector Index: Implement Chroma (or FAISS) index creation, similarity-search configuration, and index validation. [4 Marks]
- Subtask 3.2.3 — Implement Retrieval Workflow: Implement naive similarity retrieval, context selection, and prompt augmentation. [4 Marks]

Task 3.3 — Evaluate Solution V1 [Deliverable: Solution V1 RAG Evaluation Notebook] [10 Marks]

- Subtask 3.3.1 — Execute Automated Testing: Evaluate generated outputs across the full held-out test set using automated testing. [3 Marks]
- Subtask 3.3.2 — Measure Format Adherence: Validate syntax and report results. [2 Marks]
- Subtask 3.3.3 — Measure Execution Success: Evaluate semantic similarity against SOP-grounded references (ROUGE/BLEU). [2 Marks]
- Subtask 3.3.4 — Measure Output Consistency: Evaluate deterministic behaviour using repeated inference. [1 Mark]
- Subtask 3.3.5 — Measure Hallucination Frequency: Evaluate unsupported claims, invalid references, and prompt violations. [2 Marks]

Task 3.4 — Analyse Retrieval Impact [Deliverable: Solution V1 RAG Evaluation Notebook] [4 Marks]

- Subtask 3.4.1 — Compare Baseline and Solution V1: Compare functional correctness, consistency, and hallucination frequency with quantified improvement percentages. [4 Marks]

Stage 4 — Solution V2 (Fine-Tuned Retrieval-Assisted Generation) [25 Marks]

Task 4.1 — Configure Fine-Tuning Pipeline [Deliverable: Fine-Tuning Pipeline Notebook] [5 Marks]

- Subtask 4.1.1 — Configure PEFT Framework: Configure LoRA or QLoRA, adapter settings, target modules, and training environment. [3 Marks]
- Subtask 4.1.2 — Configure Training Parameters: Define learning rate, batch size, steps/epochs, optimisation strategy (AdamW), and quantisation settings. [2 Marks]

Task 4.2 — Train Fine-Tuned Model [Deliverable: Fine-Tuning Pipeline Notebook] [7 Marks]

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

- Subtask 4.2.1 — Execute Fine-Tuning: Launch training; monitor train and validation metrics at regular intervals; implement early stopping based on validation loss; save the best checkpoint by validation loss; plot train vs validation loss curves. [5 Marks]
- Subtask 4.2.2 — Save Training Outputs: Document the checkpoint and model-saving workflow and reproducibility information (training log). [2 Marks]

Task 4.3 — Integrate Fine-Tuned Model with Retrieval [Deliverable: Solution V2 Evaluation Notebook] [3 Marks]

- Subtask 4.3.1 — Integrate Fine-Tuned Model into Existing RAG Pipeline: Replace the baseline model with the fine-tuned model (acting as intent router). Reuse the retrieval workflow by mapping extracted JSON to specific vector-search strings. Validate the integrated system. [3 Marks]

Task 4.4 — Evaluate Solution V2 [Deliverable: Solution V2 Evaluation Notebook] [5 Marks]

- Subtask 4.4.1 — Re-Execute Evaluation Framework: Evaluate Format Adherence and Intent Accuracy on the held-out test split (zero leakage) and an adversarial subset derived via regex filtering. Evaluate the final synthesis using ROUGE/BLEU. [3 Marks]
- Subtask 4.4.2 — Analyse Fine-Tuning Impact: Compare Solution V1 and Solution V2, quantifying the improvement attributable to fine-tuning. [2 Marks]

Task 4.5 — Perform Comparative Analysis [Deliverable: Comparative Analysis Report PDF] [5 Marks]

- Subtask 4.5.1 — Compare All Solution Versions: Compare Baseline, Solution V1, and Solution V2 using all evaluation metrics with SOP-grounded references. [3 Marks]
- Subtask 4.5.2 — Document Findings and Recommendations: Document key findings, limitations, recommendations, and future work. [2 Marks]

Deliverables

1. Project_Proposal_and_Methodology.pdf
2. Data_Understanding_and_EDA.ipynb
3. Data_Preparation.ipynb
4. Baseline_Model_Evaluation.ipynb
5. RAG_Implementation.ipynb
6. Solution_V1_RAG_Evaluation.ipynb
7. Fine_Tuning_Pipeline.ipynb
8. Solution_V2_FineTuned_RAG_Evaluation.ipynb
9. Comparative_Analysis_Report.pdf

Note: Model artefacts (.bin, .safetensors, checkpoint/adapter folders, vector DB directories) are NOT required. Evidence is captured through notebook implementation, training logs, loss curves, configuration records, and reproducibility documentation.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Tools, Libraries, and Technology Stack

The following tools and libraries are recommended for implementing this assignment.

Programming Environment

- Python 3.10+
- Jupyter Notebook or Google Colab
- GPU Runtime (recommended)

Data Processing and Analysis

- pandas
- numpy
- scikit-learn
- matplotlib
- Seaborn

Large Language Model Development

- transformers
- datasets
- torch
- accelerate
- peft
- bitsandbytes

Retrieval and Vector Search

- sentence-transformers
- ChromaDB (recommended) or FAISS
- LangChain document loaders and text splitters

Recommended Models

- Qwen/Qwen2.5-1.5B-Instruct (recommended)
- TinyLlama/TinyLlama-1.1B-Chat-v1.0
- meta-llama/Llama-3-8B-Instruct (optional)

Recommended Embedding Model

- sentence-transformers/all-MiniLM-L6-v2

Evaluation Libraries

- rouge-score
- nltk
- sentence-transformers
- sklearn.metrics

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

Implementation Guidance

You are advised to follow the implementation sequence below.

1. Use a consistent configuration throughout the project:
   - Same MODEL_ID
   - Same embedding model
   - Same dataset partitions
   - Same evaluation datasets

2. Establish the baseline system before implementing retrieval and fine-tuning.

3. Validate each stage independently before integration:
   - Dataset preparation
   - Baseline evaluation
   - Retrieval implementation
   - Fine-tuning pipeline
   - Hybrid RAG evaluation

4. Save intermediate artefacts generated during each notebook execution, including:
   - Dataset partitions
   - Chroma database
   - Evaluation outputs
   - Training logs
   - Model checkpoints
   - Visualisations and reports

5. Perform evaluations only on the held-out test split and maintain complete separation between training, validation, and testing datasets.

6. Document all assumptions, configuration choices, limitations, and experimental observations.

Common Mistakes and Troubleshooting

Data Understanding and Preparation

- Failure to remove near-duplicate prompts.
- Inconsistent JSON target formatting.
- Dataset leakage across train, validation, and test splits.
- Severe token truncation due to inappropriate sequence lengths.

Baseline Evaluation

- Using temperature=0.0 with do_sample=False.
- Loading models without 4-bit quantisation, causing memory issues.

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

- Using different MODEL_ID values across notebooks.

Retrieval Implementation

- Using different embedding models across experiments.
- Incorrect or incomplete Chroma database creation.
- Retrieving irrelevant policy documents due to poor query construction.
- Modifying retrieval configurations between evaluation stages.

Fine-Tuning

- Training without validating dataset formatting.
- Failure to monitor validation loss.
- Failure to save the best checkpoint.
- Excessive sequence lengths resulting in out-of-memory errors.

Hybrid RAG Evaluation

- Router outputs containing explanatory text instead of valid JSON.
- JSON parsing failures.
- Evaluation on inconsistent datasets or configurations.
- Comparing systems using different inference parameters.

General Recommendation

Validate each notebook independently before proceeding to the next stage and preserve all intermediate artefacts generated during execution.

Disclaimer: *All content and material on the upGrad website is copyrighted material, belonging to either upGrad or its bonafide contributors, and is purely for the dissemination of education. You are permitted to access, print, and download extracts from this site purely for your own education only and on the following basis:*

- *You can download this document from the website for self-use only.*
- *Any copies of this document, in part or full, saved to disc or to any other storage medium, may be used for subsequent, self-viewing purposes or to print an individual extract or copy for noncommercial personal use only.*
- *Any further dissemination, distribution, reproduction, or copying of the content of the document herein, or the uploading thereof on other websites, or use of the content for any other commercial/unauthorized purposes in any way that could infringe the intellectual property rights of upGrad or its contributors, is strictly prohibited.*
- *No graphics, images, or photographs from any accompanying text in this document will be used separately for unauthorized purposes.*
- *No material in this document will be modified, adapted, or altered in any way.*

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.

upGrad

- *No part of this document or upGrad content may be reproduced or stored on any other website or included in any public or private electronic retrieval system or service without upGrad's prior written permission.*
- *Any rights not expressly granted in these terms are reserved.*

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.